

LAPORAN SOFTWARE QUALITY ASSURANCE

METODE WHITEBOX – LOOP TESTING



Anggota kelompok:

20231310046
20231310047

M. Irvan Alfiansyah
M. Nur Yanfa

PROGRAM STUDI TEKNIK INFORMATIKA
UNIVERSITAS KEBANGSAAN REPUBLIK INDONESIA

*Jln. Terusan Halimun No. 37 (Pelajar Pejuang 45) Lingkar Selatan Kec. Lengkong
kota Bandung Jawa Barat 40263*

1. DASAR TEORI LOOP TESTING

Loop Testing adalah metode white box testing yang digunakan untuk memeriksa struktur perulangan (loop) dalam program secara menyeluruh. Pengujian ini memastikan setiap loop bekerja dengan benar pada berbagai kondisi iterasi — dari kondisi paling minimal hingga kondisi data campuran.

Kondisi iterasi yang diuji dalam Loop Testing:

Kondisi Loop	Keterangan	Contoh pada Checkout ACID
Zero Iteration	Loop tidak dijalankan karena data kosong	Cart kosong — tidak ada item untuk diproses
One Iteration	Loop berjalan tepat satu kali	Cart berisi tepat satu produk
Multiple Iteration	Loop berjalan beberapa kali	Cart berisi beberapa item produk berbeda
Mixed Condition	Loop berisi data valid dan invalid	Batch order: ada order PAID milik supplier, ada PENDING, ada milik supplier lain
Boundary Loop	Jumlah data dekat batas bisnis	Quantity dan stok produk pada skenario checkout

Loop Testing relevan untuk Checkout ACID karena satu transaksi checkout memproses banyak item sekaligus. Kesalahan pada struktur loop dapat mengakibatkan order tidak lengkap, total harga salah, pengurangan stok parsial, atau pemrosesan order supplier yang tidak berwenang.

2. RUANG LINGKUP PENGUJIAN

Pengujian Loop Testing mencakup dua area loop utama yang berhubungan langsung dengan alur checkout dan proses pasca-checkout:

- Loop cart item pada proses checkout (order_repository.go — CheckoutTransaction)
- Loop order/supplier pada proses batch order supplier (order_usecase.go — BatchProcessSupplierOrders)

ID Black-Box	Skenario Checkout ACID	Dicakup	Alasan
CHK-01	Checkout berhasil	Ya	Cart berisi item harus diproses lengkap tanpa ada yang hilang
CHK-02	Stok tidak mencukupi	Ya (source)	Loop repository memvalidasi stok setiap item satu per satu
CHK-03	Race condition checkout bersamaan	Ya (source)	Loop melakukan FOR UPDATE lock pada setiap produk/varian
CHK-04	Keranjang kosong	Ya	Zero item tidak boleh masuk ke dalam transaction database

ID Black-Box	Skenario Checkout ACID	Dicakup	Alasan
CHK-07	Atomicity stok dan order	Ya	Semua item diproses dalam satu transaction — tidak boleh parsial
CHK-08	Status order PENDING	Ya (source)	Setelah loop sukses memproses semua item, order dibuat berstatus PENDING

BatchProcessSupplierOrders tidak berada tepat di endpoint checkout, tetapi merupakan proses terkait yang terjadi setelah pembayaran dikonfirmasi. Loop pada fungsi ini diuji sebagai tambahan white-box untuk memastikan pemrosesan banyak order tidak salah filter.

3. SOURCE CODE LOOP YANG DIUJI

3.1 Loop Cart Item pada Repository — CheckoutTransaction

Loop ini memproses seluruh item keranjang dalam satu transaksi database. Setiap iterasi melakukan lock produk/varian, validasi stok, pengurangan stok, perhitungan harga, dan pembentukan order item.

```

for _, item := range cartItems {
    // — Step 1: Lock produk FOR UPDATE —————
    var product domain.Product
    if err := tx.Clauses(clause.Locking{Strength: "UPDATE"}).
        Where("id_product = ?", item.ProductID).First(&product).Error; err !=
nil {
        return errors.New("produk " + item.ProductID + " tidak ditemukan")
    }
    priceAtPurchase := product.Price

    // — Step 2: Cek varian (jika ada) —————
    if item.VariantID != nil {
        var variant domain.ProductVariant
        if err := tx.Clauses(clause.Locking{Strength: "UPDATE"}).
            Where("id_variant = ?", *item.VariantID).First(&variant).Error;
err != nil {
            return errors.New("varian produk tidak ditemukan")
        }
        if variant.Stock < item.Quantity {
            return fmt.Errorf("stok varian '%s' tidak mencukupi. Stok
tersisa: %d",
                variant.NameLabel, variant.Stock)
        }
        variant.Stock -= item.Quantity
        tx.Save(&variant)
    }
}

```

```

        priceAtPurchase = variant.Price
    } else {
        // — Step 3: Validasi & kurangi stok produk —————
        if product.Stock < item.Quantity {
            return fmt.Errorf("stok produk '%s' tidak mencukupi. Stok
tersisa: %d",
                product.Name, product.Stock)
        }
        product.Stock -= item.Quantity
        tx.Save(&product)
    }

    // — Step 4: Akumulasi total dan order item —————
    totalAmount += priceAtPurchase * float64(item.Quantity)
    orderItems = append(orderItems, domain.OrderItem{ ... })
}

```

3.2 Guard Cart Kosong pada Usecase

Guard ini berada sebelum loop repository dipanggil. Cart kosong langsung ditolak di level usecase — transaksi database tidak pernah dimulai.

```

cartItems, err := u.cartRepo.FindByUserID(userID)
if err != nil {
    return nil, errors.New("gagal memuat keranjang belanja")
}

// Guard zero iteration — cart kosong ditolak sebelum repository dipanggil
if len(cartItems) == 0 {
    return nil, errors.New("keranjang belanja anda kosong. tidak bisa
checkout")
}

order, err := u.orderRepo.CheckoutTransaction(userID, cartItems, voucherCode)

```

3.3 Loop Batch Supplier Orders — BatchProcessSupplierOrders

Loop ini memproses daftar order untuk menentukan mana yang valid bagi supplier tertentu. Setiap order dicek kepemilikan supplier-nya dan status pembayarannya sebelum diproses lebih lanjut.

```

validOrderIDs := []string{}
now := time.Now()

for _, order := range orders {

```

```

// — Loop inner: cek kepemilikan supplier —————
isOwnedBySupplier := false
for _, item := range order.Items {
    if item.Product != nil && item.Product.SupplierID == supplierID {
        isOwnedBySupplier = true
        break // short-circuit: cukup satu item cocok
    }
}

// — Filter: harus milik supplier DAN berstatus PAID —————
if isOwnedBySupplier && order.Status == "PAID" {
    validOrderIDs = append(validOrderIDs, order.ID)
    if u.auditLogRepo != nil {
        _ = u.auditLogRepo.Insert(&domain.AuditLog{ ... })
    }
}

// Guard: tidak ada order valid
if len(validOrderIDs) == 0 {
    return errors.New("tidak ada pesanan yang valid untuk diproses")
}

return u.orderRepo.BatchUpdateStatus(validOrderIDs, "PROCESSED")

```

4. IDENTIFIKASI RISIKO LOOP

Sebelum merancang test case, risiko yang dapat terjadi akibat kesalahan struktur loop diidentifikasi secara sistematis:

ID Risiko	Area Loop	Risiko	Dampak Jika Terjadi
RL-01	Cart Item	Cart kosong tetap diproses	Order kosong dapat dibuat — melanggar business rule
RL-02	Cart Item	Item terakhir tidak ikut diproses	Order item tidak lengkap, total harga salah
RL-03	Cart Item	Item diproses ganda (double processing)	Total harga dan stok salah — kerugian finansial
RL-04	Cart Item	Variant item tidak diproses	Stok varian tidak berkurang — overselling varian
RL-05	Cart Item	Error stok satu item tidak rollback keseluruhan	Transaksi parsial — stok berkurang sebagian tanpa order terbentuk
RL-06	Batch Supplier	Order supplier lain ikut diproses	Pelanggaran otorisasi — supplier A memproses order supplier B
RL-07	Batch Supplier	Order status PENDING ikut diproses	Alur pembayaran rusak — order diproses sebelum dibayar
RL-08	Batch Supplier	Audit log dibuat untuk order invalid	Jejak audit tidak akurat — menyulitkan investigasi

5. RANCANGAN TEST CASE

Lima test case dirancang untuk mencakup seluruh kondisi iterasi yang relevan — zero, one, multiple, mixed, dan boundary — sesuai metodologi Loop Testing:

ID	Jenis Loop	Kondisi Iterasi	Data Uji	Expected Result	
LP-CHK-01	Cart Loop	Zero Iteration	cartItems = [] (kosong)	Checkout error; CheckoutTransaction tidak dipanggil	PASS
LP-CHK-02	Cart Loop	One Iteration	1 cart item (prod-1, qty=1)	Repository dipanggil 1x; 1 item diteruskan	PASS
LP-CHK-03	Cart Loop	Multiple Iteration	3 cart item (prod-1 qty=1, prod-2 qty=2, prod-3 qty=3)	Repository dipanggil 1x; 3 item diteruskan lengkap	PASS
LP-CHK-04	Batch Supplier	Mixed Condition	4 order: 2 valid (PAID+milik), 1 PENDING, 1 supplier lain	Hanya 2 order valid diproses; 2 lainnya difilter	PASS
LP-CHK-05	Batch Supplier	No Valid Order	1 order PENDING (bukan PAID)	Return error; BatchUpdateStatus tidak dipanggil	PASS

6. SOURCE CODE AUTOMATED TEST

6.1 Test Loop Cart Item — TestLoopTesting_CheckoutCartItemCounts

Test table-driven ini mencakup tiga kondisi iterasi sekaligus: zero (cart kosong), one (1 item), dan multiple (3 item). Setiap kondisi memverifikasi jumlah pemanggilan repository dan ada/tidaknya error.

```
func TestLoopTesting_CheckoutCartItemCounts(t *testing.T) {
    tests := []struct {
        name          string
        items          []domain.CartItem
        wantRepoCalls int
        wantErr        bool
    }{
        {
            name:          "0 item",
            items:         []domain.CartItem{},
            wantRepoCalls: 0,
            wantErr:        true, // harus error - zero iteration
        },
        {
            name: "1 item",
            items: []domain.CartItem{
                {ID: "cart-1", ProductID: "prod-1", Quantity: 1},
            },
            wantRepoCalls: 1,
        },
        {
            name: "3 items",
            items: []domain.CartItem{
                {ID: "cart-1", ProductID: "prod-1", Quantity: 1},
                {ID: "cart-2", ProductID: "prod-2", Quantity: 2},
                {ID: "cart-3", ProductID: "prod-3", Quantity: 3},
            },
            wantRepoCalls: 1, // dipanggil 1x dengan 3 item sekaligus
        },
    }
    // ... setup mock, run, assert ...
}
```


6.2 Test Loop Batch Supplier — TestLoopTesting_BatchProcessSupplierOrders

Test ini menyiapkan 4 order dengan kondisi campuran (mixed condition) untuk memverifikasi filter supplier dan filter status berjalan secara bersamaan dalam satu loop.

```
func TestLoopTesting_BatchProcessSupplierOrders(t *testing.T) {
    supplierID      := "supplier-loop"
    otherSupplierID := "supplier-other"

    orders := []domain.Order{
        { // Valid: PAID + milik supplier
          ID: "order-valid-1", Status: "PAID",
          Items: []domain.OrderItem{
              {Product: &domain.Product{SupplierID: supplierID}},
          },
        },
        { // Invalid: PENDING (belum dibayar)
          ID: "order-wrong-status", Status: "PENDING",
          Items: []domain.OrderItem{
              {Product: &domain.Product{SupplierID: supplierID}},
          },
        },
        { // Invalid: PAID tapi milik supplier lain
          ID: "order-other-supplier", Status: "PAID",
          Items: []domain.OrderItem{
              {Product: &domain.Product{SupplierID: otherSupplierID}},
          },
        },
        { // Valid: PAID + milik supplier
          ID: "order-valid-2", Status: "PAID",
          Items: []domain.OrderItem{
              {Product: &domain.Product{SupplierID: supplierID}},
          },
        },
    }

    // Expected: hanya order-valid-1 dan order-valid-2 yang diproses
}
```

6.3 Test No Valid Order —

TestLoopTesting_BatchProcessSupplierOrders_NoValidOrder

Test ini memverifikasi guard akhir loop: ketika tidak ada order yang lolos filter, sistem harus mengembalikan error dan tidak memanggil BatchUpdateStatus.

```
func TestLoopTesting_BatchProcessSupplierOrders_NoValidOrder(t *testing.T) {
    // Satu order yang ada berstatus PENDING – tidak lolos filter PAID
    orders := []domain.Order{
        {
            ID: "order-pending", Status: "PENDING",
            Items: []domain.OrderItem{
                {Product: &domain.Product{SupplierID: "supplier-loop"}},
            },
        },
    }
    // Expected: return error, BatchUpdateStatus tidak pernah dipanggil
}
```

7. LANGKAH PENGUJIAN

Pengujian dilakukan mengikuti urutan langkah berikut:

1. Menyiapkan mock cart repository dengan variasi jumlah item (0, 1, dan 3 item).
2. Menjalankan fungsi Checkout untuk masing-masing kondisi dan mencatat respons.
3. Mengamati jumlah pemanggilan CheckoutTransaction pada mock repository.
4. Mengamati jumlah cart item yang diteruskan ke repository pada setiap kondisi.
5. Menyiapkan mock order repository berisi 4 order dengan kombinasi status dan supplier.
6. Menjalankan BatchProcessSupplierOrders dan mencatat order ID yang masuk validOrderIDs.
7. Memastikan hanya order yang memenuhi dua syarat (PAID + milik supplier) yang masuk BatchUpdateStatus.
8. Memverifikasi audit log hanya dibuat untuk order yang valid.
9. Menjalankan test no-valid-order dan memverifikasi error dikembalikan tanpa memanggil BatchUpdateStatus.

8. HASIL PENGUJIAN AKTUAL

8.1 Command Eksekusi

```
cd backend-go
go test ./internal/usecase -run TestLoopTesting -v
```

8.2 Output Lengkap

```
=== RUN    TestLoopTesting_CheckoutCartItemCounts
=== RUN    TestLoopTesting_CheckoutCartItemCounts/0_item
=== RUN    TestLoopTesting_CheckoutCartItemCounts/1_item
=== RUN    TestLoopTesting_CheckoutCartItemCounts/3_items
--- PASS: TestLoopTesting_CheckoutCartItemCounts (0.00s)
    --- PASS: TestLoopTesting_CheckoutCartItemCounts/0_item (0.00s)
    --- PASS: TestLoopTesting_CheckoutCartItemCounts/1_item (0.00s)
    --- PASS: TestLoopTesting_CheckoutCartItemCounts/3_items (0.00s)
=== RUN    TestLoopTesting_BatchProcessSupplierOrders
--- PASS: TestLoopTesting_BatchProcessSupplierOrders (0.00s)
=== RUN    TestLoopTesting_BatchProcessSupplierOrders_NoValidOrder
--- PASS: TestLoopTesting_BatchProcessSupplierOrders_NoValidOrder (0.00s)
PASS
ok      github.com/nuryanfa/e-commerce-sqa/internal/usecase
```

Seluruh 5 sub-test PASS | Durasi eksekusi: 0.00s | Status keseluruhan: PASS

8.3 Ringkasan Hasil per Sub-Test

Nama Sub-Test	Kondisi Iterasi	Durasi	Status
<i>TestLoopTesting_CheckoutCartItemCounts/0_item</i>	Zero	0.00s	PASS
<i>TestLoopTesting_CheckoutCartItemCounts/1_item</i>	One	0.00s	PASS
<i>TestLoopTesting_CheckoutCartItemCounts/3_items</i>	Multiple	0.00s	PASS
<i>TestLoopTesting_BatchProcessSupplierOrders</i>	Mixed	0.00s	PASS
<i>TestLoopTesting_BatchProcessSupplierOrders_NoValidOrder</i>	No Valid	0.00s	PASS

9. ANALISIS PER TEST CASE

ID	Hasil Aktual	Risiko yang Dimitigasi	Analisis
LP-CHK-01	Cart kosong menghasilkan error; CheckoutTransaction tidak dipanggil	RL-01	Zero iteration aman; tidak ada order kosong yang dapat terbentuk
LP-CHK-02	Cart 1 item memanggil repository 1 kali dan membawa tepat 1 item	RL-02 / RL-03	One iteration valid; tidak ada item yang hilang atau terduplikasi
LP-CHK-03	Cart 3 item memanggil repository 1 kali dan membawa tepat 3 item	RL-02 / RL-03	Multiple iteration tidak menghilangkan item; total item tetap konsisten
LP-CHK-04	Dari 4 order campuran, hanya order-valid-1 dan order-valid-2 yang diproses	RL-06 / RL-07 / RL-08	Filter ganda (supplier + status) bekerja benar; order invalid tidak bocor
LP-CHK-05	Order PENDING saja menghasilkan error; BatchUpdateStatus tidak dipanggil	RL-07	Guard akhir loop berfungsi; tidak ada batch update pada kondisi kosong

10. ANALISIS ACID HASIL LOOP TESTING

Prinsip ACID	Status	Bukti Loop Testing	Penjelasan
Atomicity	Terpenuhi	Cart kosong tidak masuk transaction; cart valid diproses sebagai satu set (LP-CHK-01, LP-CHK-03)	Tidak ada transaction parsial — semua item masuk atau tidak sama sekali
Consistency	Terpenuhi	Jumlah item input sama dengan jumlah item yang diteruskan ke repository (LP-CHK-02, LP-CHK-03)	Tidak ada item yang hilang selama traversal loop — data konsisten
Isolation	Terpenuhi (design)	Loop repository mengunci produk/varian satu per satu dengan FOR UPDATE (source analysis)	Stok tiap item terlindungi dari race condition pada checkout bersamaan
Durability	Terpenuhi	Batch supplier membuat audit log untuk setiap order valid yang diproses (LP-CHK-04)	Perubahan status order memiliki jejak audit yang tersimpan permanen

11. TRACEABILITY MATRIX

Test Case	Kondisi Loop	Risiko Dimitigasi	Skenario Black-Box	Test Function	
LP-CHK-01	Zero	RL-01	CHK-04	<i>CartItemCounts/0_item</i>	PASS
LP-CHK-02	One	RL-02, RL-03	CHK-01	<i>CartItemCounts/1_item</i>	PASS
LP-CHK-03	Multiple	RL-02, RL-03	CHK-01, CHK-07	<i>CartItemCounts/3_items</i>	PASS
LP-CHK-04	Mixed	RL-06, RL-07, RL-08	Pasca-checkout	<i>BatchProcessSupplierOrders</i>	PASS
LP-CHK-05	No Valid	RL-07	Pasca-checkout	<i>BatchProcess_NoValidOrder</i>	PASS

12. TEMUAN DAN REKOMENDASI

ID	Temuan	Dampak	Rekomendasi	Prioritas
LP-F01	Cart kosong berhenti sebelum repository transaction dimulai	Positif: tidak ada order kosong, resource DB tidak terpakai sia-sia	Pertahankan guard <code>len(cartItems)==0</code> di usecase; jangan dipindahkan ke repository	Tinggi
LP-F02	Cart 1 dan 3 item diteruskan lengkap tanpa ada yang hilang atau ganda	Positif: total harga dan jumlah order item selalu akurat	Jadikan test <code>CartItemCounts</code> sebagai regression test wajib saat refactor checkout	Tinggi
LP-F03	Batch supplier memfilter status PAID dan kepemilikan supplier dengan benar	Positif: tidak ada order supplier lain atau belum dibayar yang bocor	Pertahankan filter ganda; tambahkan test case untuk status PROCESSED dan CANCELLED	Sedang
LP-F04	Loop stok repository diuji melalui source analysis, belum ada DB concurrency unit test	Risiko residual: race condition stok belum divalidasi pada level unit test ini	Gunakan hasil performance/endurance test dan basic path sebagai bukti tambahan; pertimbangkan integration test dengan DB nyata	Sedang

13. KESIMPULAN

Loop Testing pada fitur Checkout ACID dinyatakan **LULUS** dengan seluruh 5 test case (LP-CHK-01 s.d. LP-CHK-05) berhasil.

Pengujian telah membuktikan bahwa:

10. Cart kosong tidak masuk ke proses transaksi database — zero iteration aman.
11. Cart dengan satu item diproses benar tanpa ada item yang hilang — one iteration valid.
12. Cart dengan banyak item diteruskan secara lengkap ke repository — multiple iteration konsisten.
13. Batch supplier hanya memproses order milik supplier yang sesuai dan berstatus PAID — filter ganda bekerja.
14. Tidak ada order invalid yang lolos filter — guard akhir loop berfungsi dengan benar.
15. Audit log hanya dibuat untuk order yang valid — jejak audit akurat.

Dengan demikian, struktur perulangan pada proses checkout dan proses terkait supplier telah bekerja sesuai ekspektasi serta mendukung konsistensi transaksi Checkout ACID. Risiko residual pada concurrency stok (RL-04, RL-05) perlu dikonfirmasi lebih lanjut melalui integration test atau hasil performance/endurance test.